

BANCO DE DADOS · DATA WAREHOUSE

INTEGRAÇÃO DE 4 SISTEMAS OLTP

PROJETO ACADÊMICO · OLTP → ETL → DW UNIFICADO

Data Warehouse Dimensional e Processo ETL

Integração de 4 sistemas transacionais — Sistema de Locação de Veículos



Equipe: Tadeu Belfort Neto · Vicente Alves · João Pedro de Lacerda

PostgreSQL · Modelo Kimball

Rio de Janeiro · 2026

Regras de negócio da locadora

Reserva



Retirada



Devolução



Cobrança

01 Cliente **PF ou PJ**, com condutor individualizado (CNH e validade)

02 Veículo: **categoria, placa, chassi, acessórios** e prontuário de conservação

03 Pátios registram retirada e devolução; **movimentações entre unidades são rastreadas**

04 Datas **previstas × realizadas** impactam cobrança e métricas

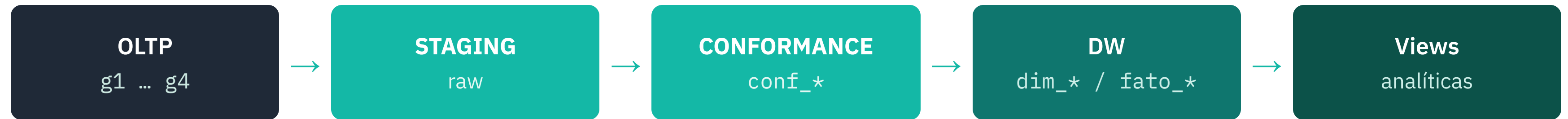
05 Fila de espera por **categoria** ou por **veículo específico** — cada regra orienta uma dimensão ou medida do DW

Quatro modelos incompatíveis

	Grupo do projeto	Grupo 2	Grupo 4	Grupo 5
Cliente	Discriminador PF/PJ	Unificado c/ docs estrangeiros	Herança PF/PJ	Discriminador
Movimentação	Tabela origem→destino	Indireto: Devolução→Vaga→Pátio	Retirada/Devolução na locação	Retirada/Devolução na locação
Endereço	Cidade inline	Entidade normalizada	Entidade normalizada	Campo TEXT

O ETL resolve essas incompatibilidades uma única vez — a análise, depois, é trivial.

Pipeline ETL em camadas



STAGING

Cópia raw da origem · auditoria ·
reprocessamento.

CONFORMANCE

Harmonização e validação dos quatro
modelos num só.

DW

Esquema estrela pronto para análise e BI.

Extração — Full Load

🕒 **Carga completa diária às 00h00** — baixo volume acadêmico

📄 **Um script por fonte** — dialetos e estruturas diferentes

📋 Staging preserva **valores originais**; registra volume e timestamp

🔑 Chave natural composta (`grupo_fonte`, `src_id`) evita colisões

tabelas stg.*

stg.cliente

stg.veiculo

stg.locacao

stg.reserva

stg.movimentacao_patio

stg.patio

stg.cobranca

Conformance — o núcleo do ETL

01 • Padronização

Booleanos, timestamps e domínios categóricos num formato comum.

02 • Cliente

Achatar herança e harmonizar **PF / PJ** numa estrutura única.

03 • Devolução

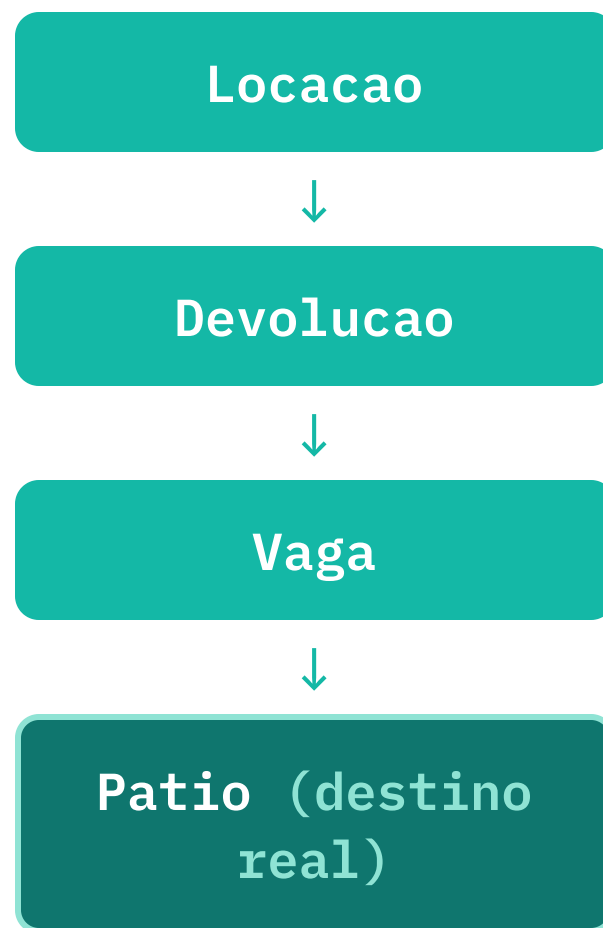
Reincorporar a devolução quando vem em tabela separada.

04 • Movimentação

Reconstrução do par origem → destino (caso Grupo 2).

A transformação mais complexa do pipeline.

Reconstrução do par origem → destino



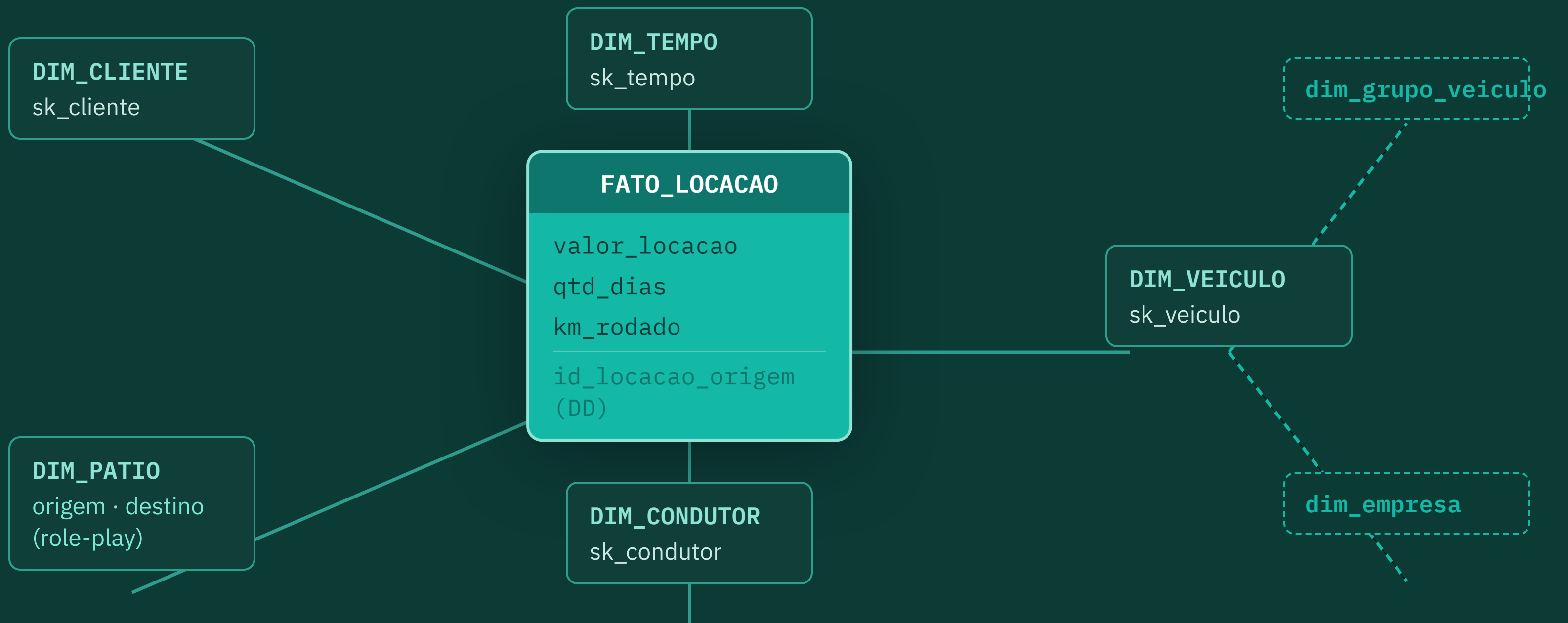
Locação → Devolução: a devolução é registrada à parte, não na locação.

Devolução → Vaga: o veículo é devolvido a uma vaga específica.

Vaga → Pátio: a vaga pertence a um pátio — o destino real.

Sem essa reconstrução, a **matriz de movimentação** não seria confiável.

Esquema estrela — dimensões qualificam o fato



Como o ETL alimenta cada tabela



Tabela no DW	Alimentada por	Regra principal
DIM_CLIENTE	conf_cliente ← stg.cliente	Achatamento PF/PJ + harmonização
DIM_VEICULO	conf_veiculo ← stg.veiculo	Ramo grupo_veiculo + empresa (snowflake)
FATO_LOCACAO	conf_locacao ← stg.locacao	Cálculo de qtd_dias e valor_locacao
FATO_VEICULO_NO_PATIO	conf_movimentacao	Reconstrução origem→destino (Grupo 2)

Perguntas que o DW responde

Pergunta de negócio	View / Fato
Qual o faturamento por mês?	vw_locacoes_por_mes
Qual o fluxo entre pátios?	vw_matriz_transicao_patios
Qual a ocupação por grupo de veículo?	vw_ocupacao_por_grupo_veiculo
Quais são os clientes mais frequentes?	vw_clientes_mais_frequentes
Quantas devoluções estão em atraso?	vw_atrasos_devolucao

Métricas e granularidade

GRÃO

Uma linha = uma **locação concluída** (retirada → devolução).

Métricas aditivas

valor_cobranca

dias_realizados

km_rodado

Dimensão degenerada

locacao_id

guardada na fato, para rastreabilidade até a fonte.

Grão único **evita ambiguidade** e garante agregações confiáveis.

Testes automatizados e reconciliação



Suíte de **testes unitários e de integração** em `04-tests`



Validações de **staging e conformance** antes da carga final



Reconciliação de contagens **staging** ↔ **DW** ao término do pipeline

GARANTIA Pipeline idempotente e determinístico.

Decisões, alternativas rejeitadas e limitações

Decisões tomadas

- ✓ **SCD Tipo 1** — foco no estado atual
- ✓ **Full load** diário — simples e determinístico
- ✓ Chave natural (`grupo_fonte`, `src_id`)

Alternativas rejeitadas

- ✗ **SCD Tipo 2 agora** — complexidade sem demanda atual
- ✗ **Carga incremental** — desnecessária no volume atual
- ✗ **Manter atributos exclusivos** — gerariam nulos sem valor



Limitação consciente: métricas ficam nulas quando a fonte não registra quilometragem.

Desafios técnicos e lições

01 **Dialetos heterogêneos** (Postgres, MySQL, ANSI) e conversão de tipos

02 **Reconstrução indireta do destino (Grupo 2)** — o maior esforço técnico

03 Decisão de **granularidade comum** para clientes e endereços

04 O **valor da staging** para auditoria e reprocessamento

Exemplos de consultas e resultados

```
SELECT ano, nome_mes, total_locacoes,
       valor_total_cobrado
FROM dw.vw_locacoes_por_mes;
```

mês	locações	valor_total_cobrado
2025-01	128	R\$ 92.400
2025-02	145	R\$ 104.300

Faturamento mensal — fato_locacao × dim_tempo.

```
SELECT origem, destino,
       percentual_transicao_origem
FROM dw.vw_matriz_transicao_patis;
```

origem	destino	%
Zona Sul	Centro	30%
Centro	Aeroporto	22%

Insight: Centro recebe ~30% das devoluções da Zona Sul.

BIG DATA · SOLUÇÃO HÍBRIDA DW + BIG DATA

OLTP → DW → BIG DATA

PROPOSTA EXECUTIVA · FROTA CONECTADA

Solução Híbrida de Data Warehouse e Big Data

Operacionalizar uma frota conectada de locadoras em escala de cidade — expansão da solução de Data Warehouse da Parte 2.



Equipe: Tadeu Belfort Neto · DRE 119034813
Vicente Alves · João Pedro de Lacerda

Databricks · Delta Lake · Kafka · PostgreSQL
Rio de Janeiro · 2026

Índice

01 Contexto e problema de negócio

02 Objetivos da solução

03 Arquitetura de referência

04 Coleta na borda: rotina × emergência

05 Ingestão de eventos (Kafka)

06 Processamento: lakehouse unificado

07 Armazenamento poliglota

08 Analytics, dashboards e IA

09 Nuvem e segurança

10 Estratégia de redução de custos

11 Extra: concierge de viagem por voz

12 Decisões-chave e conclusão

O problema de negócio

O consórcio quer automatizar e conectar a frota — tempo real, emergências e cobrança adaptativa — com um perfil de cliente bem definido:

01 Baixa capacidade de investimento e retorno rápido — **o TCO é o critério que decide.**

02 **Escala de cidade**, não de megacidade — pátios de uma mesma cidade; sem superdimensionar.

03 Frota gera **fluxo constante de dados** — exige ingestão e armazenamento robustos.

04 Já existe um **Data Warehouse (Parte 2)** a ser reaproveitado, não descartado.

Mandato da consultoria: **a stack mais simples e segura possível** dentro do orçamento.

O que a solução precisa entregar

Operação em tempo real

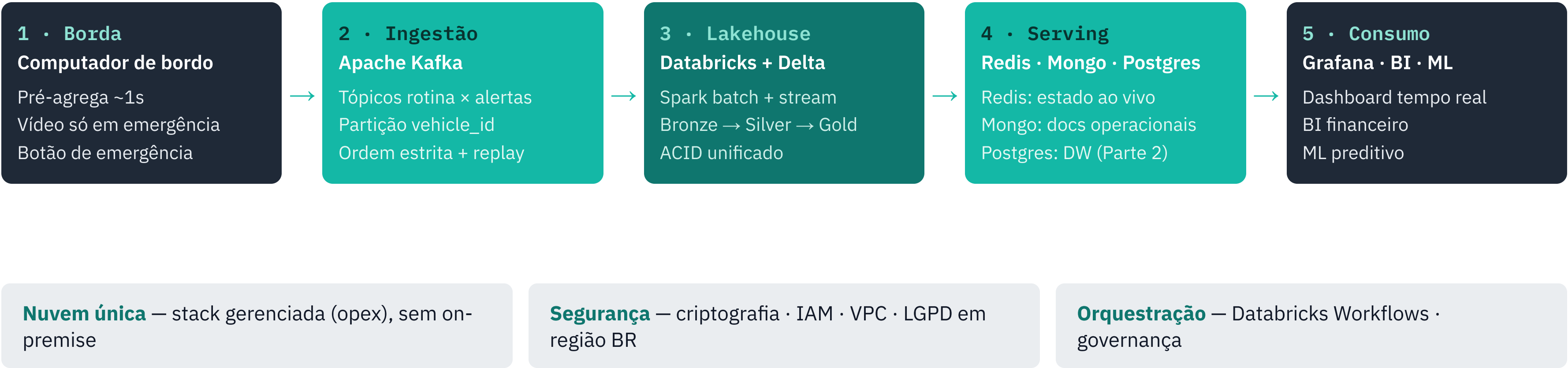
- ▶ Acompanhamento da situação da frota
- ▶ Respostas imediatas em emergências
- ▶ Mais clientes atendidos sem expandir a frota

Resultado financeiro

- ▶ Cobrança automática e adaptativa por desgaste
- ▶ Menos custo com seguradoras
- ▶ Menos prejuízo com veículo parado em manutenção

Tudo respeitando a limitação financeira — princípio que guia cada escolha.

Visão geral do pipeline



Cinco camadas, do carro ao dashboard — caminho quente (emergência) e frio (rotina) separados desde a borda.

Coleta na borda: rotina × emergência

● Rotina · caminho frio

- ▶ Telemetria compactada a cada ~1s
- ▶ Posição e situação geral do veículo
- ▶ Base da cobrança por desgaste
- ▶ Transmissão barata e espaçada

● Emergência · caminho quente

- ▶ Vídeo + dados só em gatilho (caro)
 - a) Problema interno → alerta + snapshot
 - b) Risco externo → clipe + streaming ao vivo
 - c) Botão do motorista → alerta máximo

Edge computing é a maior alavanca de custo: pré-processar a bordo elimina o gasto recorrente mais caro — a banda móvel.

Ingestão de eventos: Apache Kafka

Um barramento de eventos gerenciado desacopla a frota dos consumidores e absorve picos sem perder dados.

Tópicos separados — alerta de emergência nunca fica na fila atrás da telemetria.

Partição por vehicle_id — ordenação estrita por veículo + timestamp sob carga.

Tolerância a falhas e replay — reprocessamento histórico sem fonte adicional.

Gerenciado (MSK / Confluent) — zero servidor Kafka para a equipe operar.

Por que não REST direto ao banco: gargalo de concorrência e perda de eventos no pico.

Lakehouse unificado — um motor, não três

Databricks + Spark Structured Streaming — um motor para batch e stream, um só código.

Delta Lake medalhão — Bronze → Silver (anonimizado) → Gold, com ACID entre batch e stream.

Modelo Kappa — reprocesso por replay; sem a duplicação de lógica do Lambda.

Bronze



Silver



Gold

✓ **Mantido**

Spark (batch + stream) sobre Delta Lake

✗ **Descartado • right-sizing**

Hadoop/MapReduce — legado

Flink — latência sub-segundo não se justifica em escala de cidade

Menos peças, menos código, menos a operar — sem perder capacidade analítica.

Poliglota — cada dado no lugar certo

Tecnologia	Papel	Por quê
Redis	Estado ao vivo da frota + cache de rotas	Leitura sub-milissegundo para o tempo real
MongoDB Atlas	Documentos: viagens, sinistros, inspeção	Schema flexível; gerenciado
Delta Lake	Histórico analítico de telemetria	Barato, escalável e com ACID
PostgreSQL (DW)	Financeiro e cobrança — reuso da Parte 2	ACID estrito; zero licença nova

Descartamos Cassandra: para escala de cidade seria superdimensionar — o Delta Lake cobre a telemetria com menos custo.

Analytics, dashboards e IA

Tempo real

Grafana

- ▶ Frota: ativos, circulação, manutenção
- ▶ Viagens em andamento e emergências
- ▶ Alimentado por Redis / streaming

Negócio

BI sobre o DW

- ▶ Reservas, locações, cobranças, financeiro
- ▶ Metabase (open-source) ou Power BI
- ▶ Cobrança adaptativa pós-devolução

Inteligência

ML

- ▶ Previsão de falhas mecânicas
- ▶ Previsão de demanda por região
- ▶ Databricks ML / MLflow

Cobre os quatro tipos de análise — descritiva, diagnóstica, preditiva e prescritiva — sobre uma base única.

Nuvem única e segurança

Por que nuvem única (sem híbrido)

- ▶ Opex, não capex — sem comprar hardware
- ▶ Sem equipe de data center para manter
- ▶ Elimina a sincronização on-prem ↔ cloud
- ▶ Stack portátil entre nuvens

Como mantemos a segurança

- ▶ Criptografia em repouso e em trânsito
- ▶ IAM de menor privilégio + VPC
- ▶ LGPD: região BR + anonimização na Silver
- ▶ Auditoria e políticas de retenção

Sigilo não exige servidor próprio: controles cloud-native + residência de dados no Brasil bastam.

Redução de custos — o fio condutor

Edge corta a banda

Só emergência sobe vídeo; elimina o maior custo recorrente.

Open-source primeiro

PostgreSQL, Kafka, Spark, Grafana; zero licença.

Batch em instâncias spot

Só o hot path fica 24/7; ML é pontual.

Armazenamento em camadas

Hot (Redis) → warm (Mongo) → cold (Delta) com TTL.

Reaproveita o DW da Parte 2

Sem reescrever nem relenciar o que já existe.

Right-sizing como princípio

Infra de cidade, não de megacidade.

Resultado: TCO baixo e ROI rápido — exatamente o que o perfil do cliente exige.

Concierge de viagem por voz

Assistente de IA embarcado que melhora a experiência da viagem — recomenda lugares, ajusta a rota, conecta-se à agenda do cliente.



Voz → **texto** → **voz**: reconhecimento de fala (STT) e síntese (TTS).

LLM + RAG sobre base local de pontos de interesse da cidade.

Ação no veículo: reprograma a rota no GPS via API.

Módulo opcional, sem inchar o pipeline central.

Alto impacto na percepção de valor; **esforço contido por ser desenho funcional**.

Por que esta é a recomendação

✓ **Simples:** um lakehouse no lugar de três motores e duas camadas.

✓ **Seguro:** criptografia, IAM, VPC e LGPD em região BR.

✓ **Econômico:** nuvem única, open-source, edge e reuso do DW.

✓ **Completo:** tempo real, histórico, cobrança adaptativa, ML e voz.

✓ **Bem dimensionado:** escala de cidade tratada como virtude, não limitação.

“Do dado operacional ao Big Data em tempo real — uma plataforma única, enxuta e segura para decidir e operar a frota.”

Tadeu Belfort Neto · Vicente Alves

João Pedro de Lacerda

github.com/tadeupires21-sketch/locadora-db